K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Fundamentals Of Operating System Lab

Lab Assignment

ENCA252

For

BCA(AI & DS)

(2026-27)

Prepared by :Preeti

Roll No : 2401201191

Submitted To : Dr. Sneh Yadav

Problem Title:

Implementation of Banker's Algorithm for Deadlock Avoidance

Problem Statement

Deadlock is a situation in operating systems where a set of processes are unable to proceed because each is waiting for resources held by others. Banker's Algorithm is a deadlock avoidance technique that ensures the system remains in a safe state. In this assignment, students will implement Banker's Algorithm and determine whether the system is in a safe state, as well as find the safe sequence of execution.

Learning Objectives

- Understand CPU scheduling concepts
- Implement FCFS and SJF algorithms
- Calculate CT, TAT, WT
- Analyze system performance

Tools/Technology Used:

- Python 3.x
- Built-in Python libraries: os, multiprocessing, time
- Linux OS (optional for simulation)

Assignment Tasks:

Task 1: System Input and Data Representation

Objective:

To understand how resources are allocated to processes in an operating system.

Description:

Students will input system data including number of processes, number of resources, allocation matrix, maximum matrix, and available resources.

Sub-Tasks:

1. Define number of processes and resources.
2. Input Allocation Matrix.
3. Input Maximum Matrix.
4. Input Available Resources.

Expected Outcome:

· Understanding of system resource allocation

· Ability to represent system state

Input :

```
# Banker's Algorithm - Complete Implementation
def input_data():
    # Task 1: System Input
    n = int(input("Enter number of processes: "))
    m = int(input("Enter number of resources: "))

    allocation = []
    print("\nEnter Allocation Matrix:")
    for i in range(n):
        allocation.append(list(map(int, input(f"P{i}: ").split())))

    maximum = []
    print("\nEnter Maximum Matrix:")
    for i in range(n):
        maximum.append(list(map(int, input(f"P{i}: ").split())))

    print("\nEnter Available Resources:")
    available = list(map(int, input().split()))

    return n, m, allocation, maximum, available


def calculate_need(n, m, allocation, maximum):
```

Output :

```
PS C:\Users\Lenovo\OneDrive\Documents\Operating System> python bankers.py
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Enter Maximum Matrix:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Enter Available Resources:
3 3 2
```

Task 2: Need Matrix Calculation

Objective:

To compute the remaining resource requirements for each process.

Description:

Need Matrix is calculated using the formula:

Need = Maximum – Allocation Sub-Tasks:

1. Calculate Need Matrix.
2. Display the Need Matrix.

Expected Outcome:

· Clear understanding of process requirements

· Ability to derive required resources

Input :

```
    # Task 2: Need Matrix Calculation
    need = []
    for i in range(n):
        row = []
        for j in range(m):
            row.append(maximum[i][j] - allocation[i][j])
        need.append(row)

    print("\nNeed Matrix:")
    for i in range(n):
        print(f"P{i}: {need[i]}")

    return need


def safety_algorithm(n, m, allocation, need, available):
```

Output :

```
Need Matrix:
P0: [7, 4, 3]
P1: [1, 2, 2]
P2: [6, 0, 0]
P3: [0, 1, 1]
P4: [4, 3, 1]
```

Task 3: Banker's Safety Algorithm Implementation

Objective:

To determine whether the system is in a safe state.

Description:

The safety algorithm checks if there exists a safe sequence in which all processes can execute without causing deadlock.

Sub-Tasks:

1. Initialize Work = Available.
2. Initialize Finish array for all processes.
3. Find a process whose Need ≤ Work.
4. Allocate resources and update Work.
5. Repeat until all processes are completed or no further allocation is possible.

Expected Outcome:

· Ability to check safe/unsafe state

· Understanding of deadlock avoidance

Input :

```
    # Task 3 & 4: Safety Algorithm + Safe Sequence
    work = available.copy()
    finish = [False] * n
    safe_sequence = []

    while len(safe_sequence) < n:
        found = False

        for i in range(n):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in range(m)):
                    print(f"\nP{i} is executing...")

                    for j in range(m):
                        work[j] += allocation[i][j]

                    safe_sequence.append(f"P{i}")
                    finish[i] = True
                    found = True

                    print(f"Updated Available (Work): {work}")

        if not found:
            break

    return finish, safe_sequence


def result_analysis(n, finish, safe_sequence):
```

Output :

```
P1 is executing...
Updated Available (Work): [5, 3, 2]

P3 is executing...
Updated Available (Work): [7, 4, 3]

P4 is executing...
Updated Available (Work): [7, 4, 5]

P0 is executing...
Updated Available (Work): [7, 5, 5]

P2 is executing...
Updated Available (Work): [10, 5, 7]
```

Task 4: Safe Sequence Determination

Objective:

To find the safe execution order of processes.

Description:

If the system is in a safe state, a sequence of execution is generated.

Sub-Tasks:

1. Store processes in safe sequence.
2. Display the safe sequence.

Expected Outcome:

- Understanding of execution order
- Visualization of safe process flow

Input :

```
    # Task 3 & 4: Safety Algorithm + Safe Sequence
    work = available.copy()
    finish = [False] * n
    safe_sequence = []

    while len(safe_sequence) < n:
        found = False

        for i in range(n):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in range(m)):
                    print(f"\nP{i} is executing...")

                    for j in range(m):
                        work[j] += allocation[i][j]

                    safe_sequence.append(f"P{i}")
                    finish[i] = True
                    found = True

                    print(f"Updated Available (Work): {work}")

        if not found:
            break

    return finish, safe_sequence


def result_analysis(n, finish, safe_sequence):
```

Output :

```
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

Task 5: Result Analysis

Objective:

To analyze system safety and interpret results.

Description:

Students will evaluate whether the system is safe or unsafe and justify their results.

Sub-Tasks:

1. Identify whether the system is in safe or unsafe state.
2. Explain the result.
3. Compare with theoretical expectations.

Expected Outcome:

- Analytical thinking
- Understanding of real-world deadlock scenario

Input :

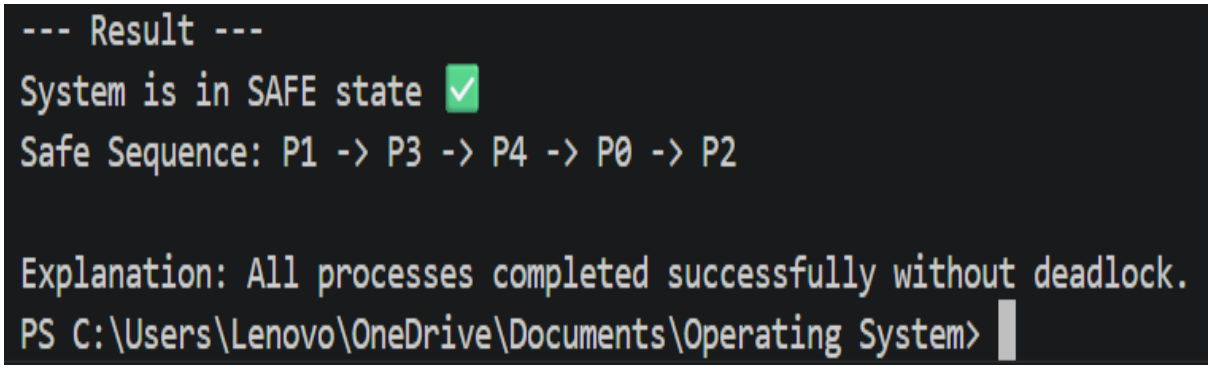
```
    # Task 5: Result Analysis
    print("\n--- Result ---")

    if all(finish):
        print("System is in SAFE state ✅")
        print("Safe Sequence:", " -> ".join(safe_sequence))
        print("\nExplanation: All processes completed successfully without deadlock.")
    else:
        print("System is in UNSAFE state ❌")
        print("\nExplanation: Deadlock may occur as not all processes could complete.")


# --------------- MAIN PROGRAM ---------------
def main():
    n, m, allocation, maximum, available = input_data()
    need = calculate_need(n, m, allocation, maximum)
    finish, safe_sequence = safety_algorithm(n, m, allocation, need, available)
    result_analysis(n, finish, safe_sequence)


# Run Program
main()
```

Output :


--- Result ---
System is in SAFE state ✅
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
Explanation: All processes completed successfully without deadlock.
PS C:\Users\Lenovo\OneDrive\Documents\Operating System>